

MatrixLang: Connecting Shape, Cost, and Numerical Contracts in a Teaching Compiler

Anonymous Author(s)

Abstract

An optimization exercise needs both a measure of saved work and a specification of the behavior it must preserve. MatrixLang is a small, straight-line teaching compiler that makes these requirements inspectable: matrix dimensions flow from type checking into a scalar-arithmetic cost model, while separate strict and algebraic modes expose the numerical consequences of rewriting. We contribute the executable artifact and a reproducible technical characterization, rather than a new optimization algorithm or a claim about student learning. Across 5,000 generated cost programs, eight optimizer configurations separate pass effects from workload design. On heterogeneous shapes, algebraic optimization reduces modeled arithmetic by a median 45.3% (IQR 0.8–79.9%) and instruction count by 5.9% (IQR 0.0–9.5%); square and elementwise controls have zero chain savings. An independent exhaustive oracle agrees on 1,250 chains. Strict optimization matches unoptimized hexadecimal output on all 2,000 generated execution programs and 15 adversarial fixtures. Algebraic optimization matches all small dyadic cases yet differs on 813 of 1,000 broad-value cases. These results support a concrete teaching artifact for studying cost, legality, and test adequacy together. They do not establish runtime speedups, universal floating-point correctness, or educational effectiveness.

Keywords: compiler construction, computing education, matrix shapes, optimization, floating-point semantics, reproducibility

1 Introduction

Compiler projects connect language syntax to executable behavior. Choosing their scope is a design problem: a language must provide meaningful semantic and optimization tasks while leaving enough implementation time for an executable compiler. Prior work explicitly examines domain-specific source languages, phase selection, and alternative targets [1, 2]. A matrix language is one useful choice because a dimension is simultaneously a typing constraint and an input to a cost calculation.

That connection alone is insufficient. An instructor can show that a different parenthesization uses fewer arithmetic operations without showing when the transformation is legal. Over real numbers, matrix multiplication is associative; over binary64 values, intermediate rounding, overflow, and signed zeros can change observable output [9, 10]. Likewise, a shorter intermediate representation need not mean less

arithmetic. These two distinctions make an optimizer a useful object of study rather than merely a collection of rewrites.

MatrixLang provides a complete executable path from a Flex scanner and Bison parser through shape analysis, three-address code (TAC), optimization, typed target instructions, and a stack virtual machine (VM). It displays each phase and a transformation log. The language omits branches, loops, functions, pointers, and dynamic shapes. This bounded scope permits inspection of a whole program as one basic block, while retaining mutable variables, literals, matrix products, scalar scaling, and transposition.

Our contribution is an educational tool with a measured account of its behavior. We investigate three questions:

1. How do modeled arithmetic reduction and TAC instruction reduction differ, and which passes account for that difference?
2. How sensitive are those reductions to the distribution of shapes and expression forms supplied by the generator?
3. What does an execution-equivalence check establish under strict and algebraic numerical contracts, and what does a small dyadic corpus miss?

The artifact answers these questions through controlled ablations, shape and expression controls, numerical counterexamples, and an independent chain-cost oracle. It also provides exercises and an assessment rubric that instructors can adapt. These are proposed uses, not observations of classroom deployment.

The study’s practical finding is that the contract and the workload must travel with an optimization result. A large modeled reduction can be legitimate in algebraic mode, absent in a square-shape control, and incompatible with exact strict output. The appropriate classroom task is therefore to explain a saving and its preconditions, not to maximize a percentage without a specification.

2 Positioning and Related Work

Compiler education. Henry describes a domain-specific language as a compiler-course vehicle [1]. Schwartzbach examines language and phase choices, including the tradeoffs of distributing a project across compiler phases [2]. These works establish that selecting a project language is a recognized curricular decision. MatrixLang does not claim that C subsets inherently lack interesting semantics or optimization. Such languages can teach control flow, aliasing, conversion, and dataflow analysis that our artifact deliberately omits.

Stella supports teaching the implementation of type systems through an extensible statically typed language [3]. Work on certified compilation takes a functional-language route to introducing correctness arguments [4]. More recent work supplies compiler intermediate-language infrastructure in Racket [5] and course-oriented translation across multiple instruction sets [6]. Our emphasis is narrower: a single matrix domain connects shape rules to an inspectable cost model and to empirical checks of numerical rewrite legality. Those connections are the artifact’s organizing principle, not an assertion that its pedagogy outperforms these alternatives.

Linear algebra. Linnea infers operand properties and generates efficient sequences of linear-algebra kernels [7]. Generalized matrix-chain optimization accounts for more operations and structure than ordinary dense products [8]. These are substantially richer optimization systems. MatrixLang implements a conventional dense chain dynamic program and small local passes to expose their operation to a learner. Static dimensions, property tracking, and cost-guided parenthesization are established mechanisms; their implementation here is not algorithmic novelty. We also do not compare our naive VM’s runtime with optimized numerical libraries.

Numerical contracts and testing. Goldberg explains why real-number identities and floating-point evaluation cannot be interchanged freely [9]. Icing formalizes configurable fast-math rewrites in a verified compiler [10]; LLVM exposes separate permissions for reassociation, signed-zero treatment, and NaN and Infinity assumptions [11]. Our two modes are a coarse, inspectable instructional interface to that established distinction. They provide neither a verification proof nor an error bound.

Csmith demonstrates validity-aware random generation and differential compiler testing [12]; NNSmith extends validity and numerical considerations to deep-learning compiler tests [13]. MatrixLang’s generator is small and author-designed. It is used to characterize this compiler, not to advance test-generation algorithms. Its unoptimized and optimized executions share a front end and VM, so agreement alone cannot validate the language implementation. The separate enumeration oracle and scalar references address specific parts of that common-mode risk. Recent educational work on curated semantic mutants likewise emphasizes testing against meaningful specification faults [14]. Our fixtures target optimizer contracts; we have no student-test or feedback study comparable to that classroom deployment.

Search scope. Positioning used live Crossref and arXiv searches across functional, structural, causal, problem, effect, inverse, and adjacent-domain formulations, followed by targeted title searches and reference inspection around

compiler education, linear-algebra compilation, and numerical correctness. Recent compiler-education preprints and the 2025–2026 SPLASH-E programs were checked separately. Broad queries often returned unrelated algebra or optimization material; an OpenAlex request was rate-limited. The companion search log records these limits. This is a scoped literature review, not a systematic review or an exhaustive novelty search. We make no priority claim about the combination.

3 Language and Compiler Design

3.1 A bounded language with static shapes

The value types are `Scalar` and `Matrix( $r, c$ )`, with positive integer dimensions known at compile time. Declared dimensions are bounded by 4,096 per axis to avoid conversion and cost overflow in the supported setting. Scalar values and matrix entries use C double; the evaluated platform uses binary64. Expressions support addition, subtraction, multiplication, negation, transposition, and constructors `identity`, `zeros`, and `ones`. Programs contain declarations, assignments, and prints. A declaration can infer its shape from its initializer. An explicitly shaped matrix without an initializer is zero-filled by the VM.

Typing and instruction selection share the same shape rules. For environment Γ , representative rules are: Writing $M_{r,c}$ for `Matrix( $r, c$ )`:

$$\frac{\Gamma \vdash A : M_{r,n} \quad \Gamma \vdash B : M_{n,c}}{\Gamma \vdash AB : M_{r,c}}, \quad (1)$$

$$\frac{\Gamma \vdash A : M_{r,c} \quad \Gamma \vdash B : M_{r,c}}{\Gamma \vdash A \pm B : M_{r,c}}, \quad \frac{\Gamma \vdash A : M_{r,c}}{\Gamma \vdash A^T : M_{c,r}}. \quad (2)$$

A scalar times a matrix preserves the matrix’s shape; scalar times scalar produces a scalar. Addition requires identical operand types, so there is no implicit scalar broadcasting. Assignments preserve a variable’s declared or inferred type. Invalid operations are rejected before code generation, with diagnostics that report operand shapes and the violated rule.

Listing 1. An inferred product and a shape-dependent chain.

```
matrix A[2,3] = {{1,2,3},{4,5,6}};
matrix B[3,2] = {{1,0},{0,1},{1,1}};
matrix P = A * B;      // Matrix(2,2)
print(P);

matrix X[100,2]; matrix Y[2,100];
matrix Z[100,2];
matrix R = X * Y * Z;
print(R);
```

In Listing 1, changing B ’s shape to $(5, 2)$ makes the first product invalid. The second fragment can be costed without constructing its matrix values. Unparenthesized multiplication is left associative.

3.2 Inspection across phases

The scanner reports classified tokens and source locations. The parser builds an AST; semantic analysis annotates it with inferred types and maintains the symbol table. TAC then represents one operation per instruction, retaining shape and source information. A source-level multiplication becomes a matrix product, matrix scaling, or scalar multiplication according to the operand types. The backend emits distinct typed VM instructions for these cases.

Command-line flags expose the token stream, AST, symbols, diagnostics, TAC, rewrite explanations, costs, target code, and execution trace. A browser demonstration additionally visualizes the scanner and the Bison-derived parsing automaton. Captured compiler output is regenerated from the executable; the browser front end is checked against the compiler over the example corpus. This supports examining the consequence of a phase, while the command-line compiler remains the authority for semantic and optimization behavior.

Generated temporary names are registered explicitly and skip user declarations. A user’s variable named `t1` therefore cannot be mistaken for a temporary. Print instructions retain their source label even when operand copies are propagated. These representation details matter because a test compares visible program output, not merely the internal numerical value.

3.3 Shapes and value properties have different roles

A square shape does not imply an identity matrix. The optimizer maintains a separate property environment for identity and zero matrices, scalar zero and one, and transpose provenance. Constructors and inspected literals establish properties; copies propagate them. Assignments invalidate facts about redefined values and transpose relationships that depend on them.

For example, if $T = A^T$ is computed and A is subsequently reassigned, replacing T^T with the current A would be unsound. The implementation must distinguish a statement about the value originally transposed from a statement about a mutable name. The reassignment fixture exercises this boundary. The property environment is a small implementation device.

4 Cost and Numerical Contracts

4.1 Two metrics with different interpretations

The arithmetic model counts conventional scalar operations. A dense product of shapes (m, n) and (n, p) costs

$$F(m, n, p) = mp(n + (n - 1)) = mp(2n - 1). \quad (3)$$

Entrywise addition, subtraction, negation, and scalar scaling cost one operation per entry. Copies, constructors, transposition, and printing have zero modeled arithmetic cost. Their allocation, movement, and I/O costs remain real but

are outside this metric. The model sums costs over live TAC instructions.

Equation 3 follows a conventional dot-product count with n multiplications and $n - 1$ additions. The VM actually initializes its accumulator to positive zero and performs n additions. Thus the model does not count every executed VM arithmetic operation: it omits *mp* initial accumulation additions per product. This choice is explicit and used consistently by the optimizer and independent oracle. No result is presented as elapsed time, machine instructions, or energy saved.

The second metric counts live TAC instructions, including copies and prints. For a program with positive baseline cost C_b , arithmetic reduction is $100(C_b - C_o)/C_b$; instruction reduction is defined analogously. Zero-cost baselines receive zero reduction rather than an undefined percentage. We summarize percentages per program, so a large product does not dominate solely because of its dimensions.

4.2 Chain ordering and its instruction-count limit

For a chain of k matrices with dimension vector $d_0, \dots, d_k$, the conventional dynamic program uses $M[i, i] = 0$ and

$$M[i, j] = \min_{i \leq s < j} \{M[i, s] + M[s + 1, j] + F(d_i, d_{s+1}, d_{j+1})\}. \quad (4)$$

Its table uses $O(k^2)$ space and $O(k^3)$ time. The implementation considers chains of at most 32 operands, retains the source order when the modeled cost does not improve, and emits the chosen tree in postorder.

Any full binary parenthesization of k operands has $k - 1$ internal product nodes. This elementary structural fact explains why the chain-only pass can reduce arithmetic without deleting a product instruction. MatrixLang reuses exactly the original product slots and preserves the final destination. It requires single-use intermediate temporaries and contiguous product slots. Without contiguity, moving a leaf earlier could cross its definition or a variable write. The compiler conservatively declines such a chain instead of performing dependence analysis. This guard limits optimization opportunities and is included in the reported artifact.

For the $(100, 2), (2, 100), (100, 2)$ chain in Listing 1, the model assigns 69,800 operations to $(XY)Z$. The alternative $X(YZ)$ costs 1,396. Each form still contains two product operations. The example illustrates a metric distinction; it is not a runtime benchmark and does not assert identical binary64 results for arbitrary matrix entries.

4.3 Strict mode

The default `-fp-strict` contract seeks to preserve printed values under the evaluated platform’s fixed default floating-point environment. It retains source product association and disables real-algebra identity and zero rewrites. It permits finite scalar constant folding, double-transpose elimination,

common-subexpression elimination (CSE), copy propagation, and dead-code elimination (DCE), subject to their data-dependence checks.

Literal and folded scalar constants are serialized with 17 significant decimal digits for binary64 round trips. Folding declines a non-finite result, leaving the original arithmetic to produce it at execution. The hand-written C sources are built with `-ffp-contract=off` and without `fast-math`. Double transposition copies entries without arithmetic; CSE preserves operand order and invalidates available expressions on relevant redefinitions. Copy propagation stops at redefinitions of either the copied name or its source. DCE works backward from print uses. These local passes repeat until a round makes no change, with a ten-round implementation ceiling; chain ordering runs once before them when enabled and permitted.

The contract is intentionally narrower than complete IEEE-754 observational equivalence. The language exposes no rounding-mode changes or floating-point exception flags. Tests do not inspect NaN payloads, allocation failures, or host-specific extended precision. Eliminating unused computations may alter unobserved exception flags or resource use. We make no proof of correctness for all accepted programs or all platforms.

4.4 Algebraic mode

The option `-fp-algebraic` permits identities over real numbers and chain reassociation. In addition to strict passes, it can replace multiplication by an identity with a copy, remove zero additions, and replace zero products with zero constructors. Shape correctness remains required. Exact binary64 output, signed-zero behavior, and NaN/Infinity propagation may change.

This distinction is operational rather than cosmetic. For a negative scalar, $x \times 0$ can produce negative zero, while the replacement literal is positive zero. A matrix containing Infinity multiplied by an identity can produce NaN through an off-diagonal $\infty \times 0$, while a copy cannot. A reassociated chain can avoid an overflowing intermediate. Algebraic mode authorizes such changes; output inequality there is reported as a contract consequence, not automatically classified as a compiler defect. No tolerance or relative-error guarantee is offered.

5 Evaluation Method

5.1 Protocol and implementation boundary

The local evaluation protocol was recorded before running seeds 101–110. Earlier seeds 1 and 2 were development data. This is an exploratory technical characterization, not an externally preregistered confirmatory experiment. The stopping rule executes every declared case and arm, and any subprocess failure, malformed result, or strict output disagreement fails the run. No generated observations were

excluded, and the compiler or generator was not tuned on the reported evaluation outcomes.

The evaluated implementation is a C11 executable built on Windows with the MSYS2/MinGW toolchain. Python orchestrates independent subprocess invocations. The environment manifest records the platform, Python version, executable hash, protocol hash, source hashes, arm flags, output hashes, and elapsed evaluation time. That elapsed time records evaluation provenance, not compiler performance. Generators, analysis scripts, numerical fixtures, raw observations, and generated LaTeX tables accompany the manuscript. We repeated all measurements after declaration-boundary and provenance-packaging repairs; these repairs did not change the workloads or optimizer configurations.

5.2 Workloads and controls

Each cost profile supplies 100 programs at each of ten seeds, for 1,000 programs per profile. Each program starts with a scalar, two runs of five compatible matrix declarations, and two additional matrices. It creates four result expressions and prints all four. Recursive expression depth is chosen from two through four; unavailable compatible operands fall back to simpler expressions. Each program uses its own stream seeded by $1,000,003 \times \text{seed} + \text{index}$.

The heterogeneous profile draws each dimension uniformly from $\{1, 2, 3, 5, 8, 12, 20, 32, 50, 80, 120, 200\}$. Its expression choices weight chain generation three times, ordinary products twice, and addition, subtraction, transpose, scaling, and variable selection once each. Chain generation attempts three through five operands, but available shapes can shorten a chain. Compatible declaration runs deliberately create chain opportunities. This is a designed workload, not a sample of naturally occurring student programs.

Four controls change that design. *Square* uses one dimension chosen from $\{2, 8, 32, 120\}$ for every axis in a program; every parenthesization then has the same modeled product cost. *Narrow* restricts dimensions to 8–12. *Balanced* retains heterogeneous shapes but assigns one equal weight to each expression category. *Elementwise* removes products and chains from the category choices, retaining addition, subtraction, transpose, scaling, and variables. Cost programs are not executed: their zero-filled input declarations provide shape information but do not constitute measured numerical workloads.

5.3 Ablation configurations

Eight configurations run on each identical cost program. *Strict* enables the default local optimizer. *Algebraic* enables all passes with the relaxed contract. *Chain only* permits chain ordering alone. *No chain* permits algebraic simplification, CSE, copy propagation, and DCE without chain ordering. Four individual-pass arms enable algebraic simplification, CSE,

copy propagation, or DCE respectively; only the algebraic-simplification arm opts into the relaxed contract. Each reported percentage uses the same program’s unoptimized baseline. The study contains 40,000 cost-arm observations.

These arms distinguish attribution from correlation. Removing chain ordering tests whether local cleanup explains the arithmetic saving; chain-only tests whether instruction deletion is necessary. Single-pass arms illustrate interactions but are not an additive decomposition: an identity can become a copy that propagation and DCE subsequently remove. Their sum need not equal the full optimizer’s reduction.

5.4 Execution comparisons and adversaries

Execution tests use dimensions 1–4, the heterogeneous expression mix, and literal-initialized matrices. The dyadic population uses small integer entries from -3 through 5 and scalar choices from $\{2, 3, 0.5, 1.5, -2\}$. The broad-value population draws signed matrix entries with mantissas uniformly between 0.1 and 9.9 and decimal exponents uniformly from -12 through 12 . Scalar choices remain the same. Each population has 1,000 programs over the ten seeds.

Each program runs unoptimized, strict optimized, and algebraic optimized. The `-exact-output` switch prints hexadecimal floating-point values, avoiding the six-significant-digit display that can conceal small rounding differences. Comparison includes every printed entry and its label. Hexadecimal equality distinguishes finite binary64 values and signed zero; textual NaN output does not distinguish payloads. All mismatch outputs and source hashes are retained.

The suite has 15 fixtures for literal precision, folded constants, cancellation, signed zero, overflow/NaN, identity multiplication, subnormals, double transpose, mutable-variable invalidation, temporary naming, and chain dependencies. Six scalar fixtures also compare packed binary64 values with independently evaluated Python references. The fixtures were designed during development and are regression tests, not randomly sampled prevalence evidence.

5.5 Independent chain-cost oracle

For each seed, the oracle generates 25 compatible dimension vectors at each chain length from three through seven operands. It recursively enumerates every binary parenthesization, without memoization, compiler cost imports, or the compiler’s dynamic-programming implementation. It calculates the conventional cost of every tree and compares the minimum with the chain-only compiler result. The 1,250 chains contain 48,750 enumerated trees in total. Instruction counts are checked before and after every chain rewrite.

This oracle validates small-chain minimum cost under the declared model. It does not validate numerical equivalence, all 32 supported operands, or every dependence guard. The adversarial noncontiguous-chain fixture tests the separate dependency boundary.

5.6 Analysis and uncertainty

For each profile and arm, we report pooled per-program median and interquartile range (IQR), per-seed summaries, and zero-saving counts. The primary paired contrast is each program’s arithmetic reduction minus its instruction reduction, in percentage points. A percentile bootstrap resamples the ten seed clusters with replacement, retaining all programs within each selected cluster, and recomputes the pooled paired median. It uses 2,000 resamples and seed 87123. This interval describes generator-stream sensitivity; ten clusters provide limited precision and do not support inference to real courses or workloads.

Wilson intervals describe synthetic mismatch frequencies under the stated sampling interpretation. The full per-seed counts are also reported so that the apparent precision of a pooled interval is not mistaken for independent classroom replication. No significance tests, multiplicity-adjusted hypotheses, or best-arm selection are used. The controls and ablations are descriptive.

6 Results

6.1 Metric disagreement and pass attribution

On heterogeneous programs, the full algebraic optimizer reduces modeled arithmetic by a median 45.3% (IQR 0.8–79.9%) and instructions by 5.9% (IQR 0.0–9.5%). The median of the paired per-program difference is 36.9 percentage points, with a seed-cluster bootstrap 95% interval of 33.3–41.6 points. This paired quantity is not the difference between the two marginal medians, nor a ratio of medians interpreted as a speedup. Figure 1 shows the distributions rather than only their centers.

Table 2 isolates the main mechanism. Chain-only has a 41.2% median arithmetic reduction and zero instruction reduction. Removing chain ordering lowers the arithmetic median to zero while retaining a 5.3% instruction median. The chain-only pass saves arithmetic on 76.7% of heterogeneous programs; the remainder are included, not discarded as uninteresting. The full algebraic configuration leaves 173 of 1,000 programs with no arithmetic saving. No arm produces a negative arithmetic or instruction reduction in this evaluation.

Single-pass medians are zero, including CSE and DCE, but some individual programs benefit. Copy propagation alone changes operands without deleting instructions or arithmetic, so both metrics remain zero by construction. The combined pipeline’s effects cannot be inferred from single-pass medians alone. The strict arm removes instructions but has a zero arithmetic median on this workload; that is consistent with disabling the reassociation mechanism responsible for most modeled savings.

6.2 The workload is part of the result

Table 1 refutes a corpus-independent interpretation of the heterogeneous result. On the square profile, chain-only saves

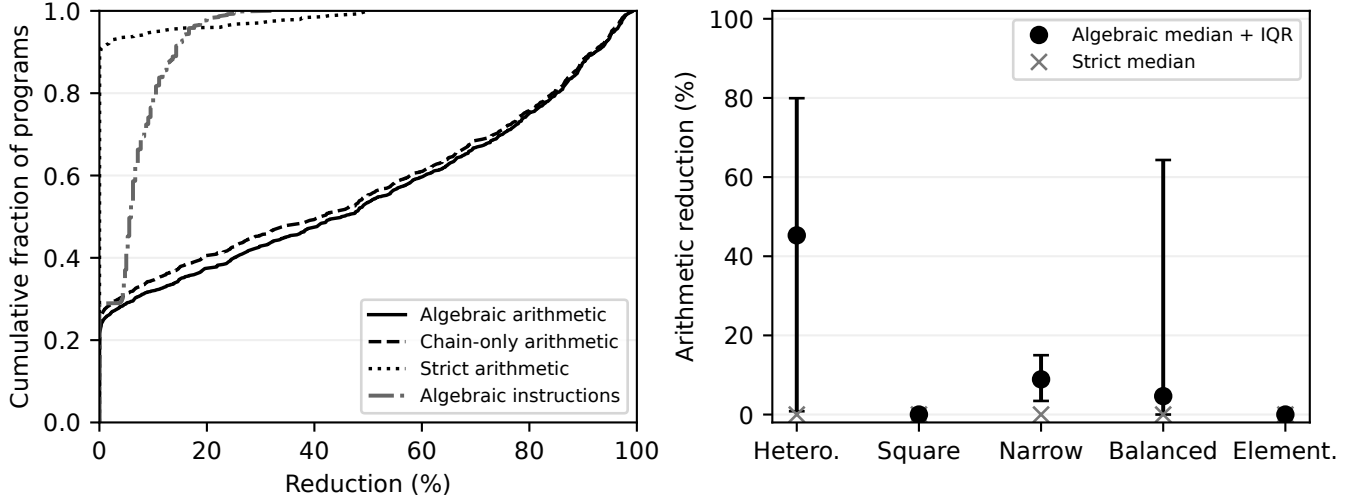


Figure 1. Left: empirical cumulative distributions of per-program reduction on 1,000 heterogeneous cost programs. Chain-only instruction reduction is identically zero and omitted. Right: algebraic arithmetic medians and IQRs across profiles; crosses mark strict medians. Error bars are distribution quartiles, not confidence intervals. The results describe modeled work, not elapsed time.

Table 1. Shape and expression sensitivity, 1,000 programs per row. Entries are median (IQR) percentage reductions. Arithmetic columns use the conventional model; the instruction column uses live TAC count for the full algebraic optimizer.

Distribution	Algebraic	Instructions	Chain only	Strict
Heterogeneous	45.3 (0.8–79.9)	5.9 (0.0–9.5)	41.2 (0.2–78.9)	0.0 (0.0–0.0)
Square	0.0 (0.0–0.0)	4.3 (0.0–5.6)	0.0 (0.0–0.0)	0.0 (0.0–0.0)
Narrow	8.9 (3.4–15.0)	4.8 (0.0–7.7)	7.7 (3.0–12.0)	0.0 (0.0–0.0)
Balanced	4.6 (0.0–64.3)	4.8 (0.0–7.7)	3.0 (0.0–62.4)	0.0 (0.0–0.0)
Elementwise	0.0 (0.0–0.0)	5.6 (0.0–10.5)	0.0 (0.0–0.0)	0.0 (0.0–0.0)

Table 2. Heterogeneous-profile ablations. Each arm has 1,000 programs. Reductions are median (IQR) percentages; the last column retains zero-saving cases.

Configuration	Arithmetic	Instructions	No arithmetic saving
Algebraic	45.3 (0.8–79.9)	5.9 (0.0–9.5)	173 / 1000
No chain	0.0 (0.0–0.0)	5.3 (0.0–7.7)	713 / 1000
Chain only	41.2 (0.2–78.9)	0.0 (0.0–0.0)	233 / 1000
Strict	0.0 (0.0–0.0)	5.3 (0.0–7.7)	713 / 1000
Algebra only	0.0 (0.0–0.0)	0.0 (0.0–0.0)	798 / 1000
Cse only	0.0 (0.0–0.0)	0.0 (0.0–0.0)	896 / 1000
Copy only	0.0 (0.0–0.0)	0.0 (0.0–0.0)	1000 / 1000
Dce only	0.0 (0.0–0.0)	0.0 (0.0–5.9)	919 / 1000

exactly zero arithmetic on every program, as predicted by equal product costs for all trees. The full algebraic optimizer still removes a median 4.3% of TAC instructions, demonstrating the converse distinction: cleanup can shorten code without a nonzero median arithmetic saving.

Restricting dimensions to 8–12 lowers full algebraic median arithmetic reduction to 8.9%; the chain-only median is

7.7%. Equal category weights lower the heterogeneous full median to 4.6%, with a broad IQR of 0.0–64.3%. The elementwise profile has zero chain savings on every case and zero median full arithmetic reduction, while instruction reduction remains 5.6%. The gap is therefore neither an invariant of matrix languages nor a property of instruction counting alone. It depends on the operations and shapes supplied.

Per-seed results in Appendix A expose variation hidden by pooling. They do not turn ten generator seeds into ten courses, implementations, or hardware platforms. The executable and declared generator population are the scope of the result.

6.3 Numerical agreement depends on the contract and test values

Strict optimization matches unoptimized hexadecimal output on all 2,000 generated execution programs: 1,000 dyadic and 1,000 broad-value cases. Both groups have zero observed disagreements; separately, a Wilson 95% interval has an upper endpoint of approximately 0.38%. This is evidence of

agreement on those synthetic populations, not a universal correctness bound.

Algebraic optimization also matches all dyadic cases, but differs on 813 of 1,000 broad-value cases (81.3%; Wilson 95% interval 78.8–83.6%). Per-seed mismatch counts range from 72 through 89 of 100. These differences include the rounding effects allowed by the contract; the comparison does not measure numerical error relative to an exact-real solution. Nor does it attribute every mismatch to reassociation rather than to another permitted algebraic rewrite.

All 15 adversarial fixtures match in strict mode, and all six independent scalar references agree. Algebraic mode differs on 6 fixtures. The chain-overflow case uses shapes (5, 1), (1, 5), (5, 1), with entries near 10^{200} , 10^{200} , 10^{-200} . The source order overflows its first product; the cheaper reassociation keeps that intermediate finite. Both are valid executions under their respective contracts. Signed-zero and identity-with-Infinity cases give smaller counterexamples to exact-algebra substitution.

The contrast between zero dyadic mismatches and frequent broad-value differences shows why a successful narrow differential test cannot establish general floating-point preservation. The artifact makes the counterexamples directly executable, so an instructor can ask what an oracle checks before discussing how many programs it checks.

6.4 Minimum modeled chain cost agrees with enumeration

For all 1,250 oracle chains, the compiler’s chosen cost equals the minimum over all enumerated parenthesizations. All before/after instruction counts agree. The comparison covers 48,750 trees and chain lengths three through seven. It independently checks the recurrence implementation’s objective on these dimensions; it does not prove global compiler correctness.

7 Using the Artifact in a Course

The intended users are students who already know basic C, syntax trees, elementary type checking, and matrix multiplication. An instructor can provide the scanner, parser, and VM as infrastructure and ask students to implement shape rules or optimizer passes. Alternatively, the phases can be constructed in sequence. The repository includes phase demonstrations and a more detailed exercise sheet with expected artifacts and a rubric. Neither delivery pattern has been evaluated with learners.

Exercise 1: make a type fact useful twice. Students infer $\text{Matrix}(r, c)$ through products and transpositions, reject incompatible dimensions, and use the same operand types to select matrix-product versus scaling instructions. Assessment examines accepted and rejected programs, diagnostic specificity, and the absence of inconsistent duplicate shape rules.

Exercise 2: explain an optimization’s objective. Students calculate both bracketings of a short chain, enumerate all trees of a longer one, and compare their answer with the dynamic-programming pass. They report arithmetic and TAC reductions separately. The square-shape control asks them to explain a legitimate zero saving, while a copy-propagation-only arm distinguishes rewritten operands from deleted work.

Exercise 3: state legality before rewriting. Students run the negative-zero, identity-with-Infinity, and chain-overflow fixtures under both modes. They must state which observations each contract preserves and why a real-number identity is insufficient evidence for strict replacement. A mutable-variable transpose case introduces invalidation; a noncontiguous chain introduces a dependence barrier.

Exercise 4: audit a reassuring test. Students compare dyadic and broad-value corpora with ordinary display output and hexadecimal output. They identify what a shared VM can conceal and add an independent reference for a selected operation. Assessment rewards a counterexample and a clearly scoped oracle, not a claim of correctness from zero observed mismatches.

The proposed rubric assigns credit to shape correctness, data-dependence reasoning, numerical-contract compliance, faithful measurement, and reproducibility. Percentage savings are evidence to explain rather than the grading objective. An instructor adopting the artifact should separately measure completion time, conceptual understanding, and usability before making claims about educational benefit.

8 Limitations and Threats to Validity

Educational validity. There are no student participants or classroom outcomes. Completion time and alternative teaching languages have not been evaluated. An executable exercise demonstrates available behavior, not learnability or improved instruction. Proposed exercises and rubric remain unvalidated. A future classroom study would need an instructor-defined curriculum, appropriate ethics review, a comparable alternative, and direct learning measures.

Workload validity. Generation emphasizes chains. The sensitivity controls show the magnitude of that choice but do not span real student programs. Generated programs rarely explore the full range of reassignment or constructor identities; fixed fixtures cover some of those cases separately. Cost and execution populations use different dimension ranges, so we do not join a cost observation to a numerical accuracy claim about the same workload.

Oracle and platform validity. Execution comparisons share scanner, parser, shape analyzer, code generator, and VM. Two paths can agree on a common error. Enumeration

checks modeled parenthesization cost, while six scalar references cover only a narrow value path. There is no independently implemented full matrix interpreter, formal semantics proof, or cross-platform validation. Hexadecimal output does not inspect NaN payloads. Compilation without contraction and the default floating-point environment are part of the tested configuration.

Metric validity. The arithmetic model ignores allocation, data movement, memory hierarchy, kernel implementation, and the VM’s initial accumulation additions. It counts only the operations it defines and is suitable for explaining that objective. A cheaper tree can require a larger intermediate or run slower. No measured runtime, memory, energy, or accuracy benefit follows from the reported cost reductions. TAC instructions are likewise not native machine instructions.

Implementation and statistical limits. Chain ordering is conservative about noncontiguous slots and single-use temporaries, and its exhaustive check stops at seven operands. Optimizer iteration is bounded by ten rounds. Ten seed clusters limit uncertainty estimation, and the bootstrap interval describes this generator rather than all possible programs. Wilson intervals assume the stated sampling view and must be read beside per-seed variation. All reported configurations share one implementation; external compiler performance baselines would answer a different question.

9 Conclusion

MatrixLang provides an inspectable connection between shape typing, modeled work, and numerical rewrite contracts in a small executable compiler. Its controlled evaluation shows that chain ordering can change arithmetic without changing product count, that shape and expression controls can largely remove the saving, and that small dyadic execution tests can conceal the numerical changes permitted by algebraic optimization. Strict mode agrees with all tested outputs, while an independent enumeration oracle checks small-chain minimum cost. The contribution is the artifact, its reproducible characterization, and proposed instructional uses. Classroom effectiveness and universal correctness remain open questions.

Artifact Availability and Research Integrity

The supplementary artifact contains the compiler, protocol, raw observations, analysis scripts, regression fixtures, and proposed exercises. An anonymous artifact access method must be supplied for venue submission. The reported evaluation uses synthetic programs and executable numerical fixtures; no human-participant study was conducted. Generative AI assistance was used for implementation and repairs, literature discovery, evaluation scripts, analysis, and manuscript drafting. Automated checks and primary-source

inspection provide the reported technical verification. Human authors retain responsibility for the manuscript; final author approval and administrative declarations are tracked in the companion submission checklist rather than asserted as completed here.

References

- [1] Tyson R. Henry. 2005. Teaching compiler construction using a domain specific language. In *SIGCSE*. <https://doi.org/10.1145/1047344.1047364>.
- [2] Michael I. Schwartzbach. 2008. Design Choices in a Compiler Course or How to Make Undergraduates Love Formal Notation. In *Compiler Construction*. https://doi.org/10.1007/978-3-540-78791-4_1.
- [3] Abdelrahman Abouneqem, Nikolai Kudasov, and Alexey Stepanov. 2024. Teaching Type Systems Implementation with Stella, an Extensible Statically Typed Programming Language. *EPTCS* 405. <https://arxiv.org/abs/2407.08089>.
- [4] Per Lindgren, Marcus Lindner, and Nils Fitinghoff. 2019. Introducing Certified Compilation in Education by a Functional Language Approach. *EPTCS* 295. <https://doi.org/10.4204/EPTCS.295.5>.
- [5] William J. Bowman. 2025. Macro-embedding Compiler Intermediate Languages in Racket. Preprint. <https://arxiv.org/abs/2509.19607>.
- [6] Francisco J. Moreno Velo and Almudena García Jurado-Centurión. 2026. URIUM: A Programming Language for a Practical Open Course on Compiler Design. Preprint. <https://arxiv.org/abs/2608.28202>.
- [7] Henrik Barthels, Christos Psarras, and Paolo Bientinesi. 2021. Linnea: Automatic Generation of Efficient Linear Algebra Programs. *ACM Transactions on Mathematical Software* 47, 3. <https://doi.org/10.1145/3446632>.
- [8] Henrik Barthels, Marcin Copik, and Paolo Bientinesi. 2018. The Generalized Matrix Chain Algorithm. In *CGO*. <https://doi.org/10.1145/3179541.3168804>.
- [9] David Goldberg. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Computing Surveys* 23, 1, 5–48. <https://doi.org/10.1145/103162.103163>.
- [10] Heiko Becker, Eva Darulova, Magnus O. Myreen, and Zachary Tatlock. 2019. Icing: Supporting Fast-Math Style Optimizations in a Verified Compiler. In *CAV*. https://doi.org/10.1007/978-3-030-25543-5_10.
- [11] LLVM Project. 2026. LLVM Language Reference Manual: Fast-Math Flags. Accessed October 5, 2026. <https://llvm.org/docs/LangRef.html#fast-math-flags>.
- [12] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI*. <https://doi.org/10.1145/1993316.1993532>.
- [13] Jiawei Liu, Jinkun Lin, Fabian Ruffy, Cheng Tan, Jinyang Li, Aurojit Panda, and Lingming Zhang. 2023. NNSmith: Generating Diverse and Valid Test Cases for Deep Learning Compilers. In *ASPLOS*. <https://doi.org/10.1145/3575693.3575707>.
- [14] Rebecca Williams Earle and Jonathan Bell. 2026. Curated Semantic Mutants: Multi-purpose Artifacts for Grading and Hinting Student Test Suites. In *SPLASH-E*. <https://doi.org/10.1145/3841644.3842688>.

A Per-Seed Results and Reconstruction

- Raw observations: results/research/cost.jsonl, results/research/execution.jsonl, results/research/oracle.jsonl, and results/research/numerics.json.
- Lineage and unrounded statistics: the environment and summary JSON files in the same directory.
- Display generation: tools/analyze_research.py.

Table 3. Heterogeneous cost profile, 100 programs per seed. Arithmetic and instruction entries are median (IQR) percentage reductions. Broad mismatches refer to algebraic versus unoptimized hexadecimal execution on a separate 100-program execution population at the same seed.

Seed	Algebraic arithmetic	Instructions	Chain only	Broad mismatches
101	41.5 (0.0–80.1)	5.6 (0.0–8.5)	33.8 (0.0–79.2)	81/100
102	38.6 (2.5–82.2)	6.2 (4.9–10.1)	30.1 (1.8–78.2)	77/100
103	47.4 (3.7–83.5)	5.6 (0.0–9.5)	46.2 (0.4–82.2)	80/100
104	47.1 (0.4–75.6)	5.9 (0.0–10.5)	41.8 (0.3–74.5)	79/100
105	44.3 (3.2–78.1)	5.6 (0.0–9.6)	38.2 (1.6–77.2)	87/100
106	39.0 (0.1–76.8)	5.7 (0.0–9.1)	36.5 (0.0–75.2)	80/100
107	55.5 (0.0–82.1)	5.6 (0.0–8.9)	55.5 (0.0–81.5)	72/100
108	53.5 (3.7–78.1)	5.3 (0.0–7.1)	48.7 (2.8–78.1)	86/100
109	51.8 (0.7–84.7)	6.2 (0.0–9.6)	48.9 (0.2–83.0)	82/100
110	37.8 (6.9–78.1)	5.9 (0.0–9.6)	33.5 (0.8–77.0)	89/100

- Full protocol: make research. Display reconstruction: make analyze. Coverage and replay checks: make verify-research.
- Proposed classroom material: docs/teaching-exercises.md.
- Author confirmations: paper/submission-checklist.md.